Candidates: 121, 299,166, 478

BAO304
Bachelor Project
Castle Stone AS
27.03.2026
31 Pages
8331 Words

Kristiania University College

URO: Guided wellness journey
Web- and Mobile App
Report

Spring 2026

Contents

1.0 Introduction

1.1 Client and background

The project is developed in collaboration with Castle Stone AS, represented by Bernard H.Bøhmer, the creator of Urometoden. Urometoden is a structured framework for emotional regulation that focuses on reflection and gradual progression over time. The method is traditionally delivered through guided sessions that help users understand and manage internal unrest.
The client's goal for this project is to explore how Urometoden could be translated into a digital format. Making it accessible for a wider audience, while preserving the method's structure, progression, and calm user experience

1.2 Project description

The project presents URO, a digital application designed to deliver Urometoden through an 8-week, structured guided journey. The solution consists of weekly content modules, which include audio exercises, reflection tasks, case-based material, and supporting content.
The project initially explored an AI-based conversational solution (ASHO) in which users would interact with language models for guidance. However, due to limitations arising from unpredictability, lack of control, and misalignment with the method's structure, the solution was to redesign it as a content-driven system.
The final product focuses on simplicity and controlled progression and is closely related to Urometoden, which is practiced in digital settings.

1.3 Limitations

The project is limited to developing a digital prototype that delivers Urometodens content through a structured 8-week journey. The following limitations apply:

- Prototype and clinical limitations: The developed solution is a prototype rather than a clinically validated tool. The application is not recognized as medical equipment and does not replace professional help regarding mental health—the effects of Urometoden in a digital form, through clinical studies within the scope of this project.
- Content production: Audio files, case studies, reflection exercises, and video content are within the project's scope. However, the material is provided by the methods authors while the project group is responsible for the digital structure and presentation, not the content itself.
- Scope imitation (pivot): Parts of the original AI-based solution (ASHO) remain in the codebase but not included in the final product. Due to the project pivot, development was redirected towards the structured Urometoden journey, and the AI-based functionality was not completed.

1.4 Research question

How can structured journeys be designed to deliver Urometodens content in a safe and accessible way, while maintaining user privacy?

The research questions of the three key dimensions:

- Structure: Refers to the pedagogical progression of the 8-week journey, in which content is curated and defined to guide users step-by-step.
- Safety and accessibility: The solution must meet the needs of users in vulnerable situations, requiring technical expertise and physical presence.
- Privacy: The system should allow users to engage with content without sharing sensitive personal information and should rely on external interactions.

1.5 Objectives

The project's objective is to develop a functional digital platform that makes Urometoden available to those seeking a self-guidance tool for emotional regulation. The targeted audience is people experiencing stress, unrest, or emotional challenges who prefer to work on the material at their own pace.
It's expected that the platform is used in a private, mobile, and self-directed context: typically at home or another safe environment, on their own device, without time pressure or external guidance. The framework needs to be calming, focused, and easy to navigate.
The market for digital tools in mental health is growing, and established companies such as Headspace and Calm have demonstrated strong demand for self-guided wellness offerings (Baumel et al., 2019). Urometoden differs from these in that it uses a method-specific, structured journey rather than a generic library.
The project's concrete contribution to Urometoden is to provide the method with a digital distribution form that preserves its pedagogical progression and enables its use at scale, without relying on individual guidance.

1.6 About the project group

The project group consists of four students with complementary skills, frontend and backend development. The group worked collaboratively throughout the project, with clear instructions divided into different responsibilities to ensure efficiency. This structure allowed the group to address both technical and design-related challenges while maintaining a shared understanding of the project goals.

1.7 Project organization

The project is organized through an agile workflow with weekly meetings. The meetings work as a combination of status update, prioritizing tasks for the coming week, and reflection on the prior week's work. The rhythm provides steady progress, addresses bottlenecks early, and is flexible enough to handle changes in scope or technical direction.
Tasks are handled on an ongoing basis and prioritized based on dependencies and risk.

Technical choices and large design decisions are documented continuously to ensure the report is built on real decisions.

2.0 Background

2.1 Urometoden explained

Urometoden is a pedagogic framework developed to help people understand and handle their unrest - a collective term that includes anxiety, stress and continuous inner unrest (Seip & Bøhmer, 2014)The method builds on the understanding that “uro” is not primarily a cognitive problem that can be “thought away”, but a bodily and emotional state that needs to be met through attentive presence and structured processing.

There are three basic principles. The first one being somatic awareness: the ability to notice how the unrest is manifesting within the body - in breath, muscle tensions, heart rhythm and other body signals - before trying to change or interpret it (Payne et al., 2015). This contrasts with the pure cognitive approach and reflects a broader development within modern trauma theory and body-oriented therapy.

The second principle is structured reflection: The user gets the opportunity to explore their own patterns through written questions and guided exercises without the need to formulate them for another person. The reflections are designed as open instructions that guide the user through a defined theme, while keeping the answers personal and private (Seip & Bøhmer, 2014).

The third principle is progression over time. The method is built on an 8-week-long journey, where each week builds upon the previous. This time structure is not arbitrary - it reflects that emotional regulation is a skill that gradually develops, and too much material at once can be overwhelming.

The digital implementation operates on these principles through four types of content: audio files (somatic work and guided meditations), case studies (recognition and normalization), reflection tasks (structured homework), and video content (pedagogical introduction to concepts).

2.2 Digital wellness tools — landscape

The market for digital tools within mental health has grown exponentially in the last decade. Established companies like Headspace and Calm have collectively millions of users and offer large libraries with meditations, sleep stories, and mindfulness exercises (Baumel et al., 2019). There are also plenty of structured CBT-based apps - like Woebot and Youper that offer cognitive-oriented self-care, often with elements of conversation-based interaction (Fitzpatrick et al., 2017).

Despite the width of these solutions share a common limitation: they are all, to an extent, generic content platforms. The user has access to a large library and is responsible for choosing what, when, and in what order to consume its contents. This can be a great solution for users with prior experience and clear goals, but less efficient for users without prior experience or who need a detailed structure to maintain their practice (Baumel et al., 2019).

A method-specific structured journey differs by offering curated content in a set order, reducing decision fatigue and increasing completion rates, a distinction explored further in section 2.4.

2.3 User needs and target group

Urometodens targeted audience is people who are experiencing unrest, stress, or challenges with emotional regulation, and prefer self-guided digital tools instead of therapy or group sessions. This group is significant: surveys indicate that a large portion of adults with mild to moderate psychological struggles won't approach professional help, often because of thresholds connected to stigma, availability, cost, or wish for privacy (Andrade et al., 2014).

2.4 Structured journeys vs open content

A central design choice in Urometoden is that the content is presented as a sequential 8-week journey, with weeks locked, active, or completed, rather than an open library where everything is available from the start. This choice carries both pedagogic and behavioral justification.

Research shows that habit formation highlights the importance of consistency and structure for new behavioral patterns to stick. Fogg's behavior model and subsequent work in "behavior design" state that small, repeated actions in a permanent context are more effective than sporadic, intense activities (Fogg, 2009). Weekly structure provides a natural rhythm: the user has something to come back to, and progression is visible.

In learning science, scaffolding is the principle that new knowledge is built on an established understanding, a basic principle(Roy D. Pea, 2004). Urometoden is developed with pedagogical progression, where previous weeks establish basic skills (such as somatic consciousness) that the latter weeks build upon. An open library would let the user jump straight into advanced exercises without the fundamentals, weakening the effect.

Engagement studies show that dropout is the primary concern for wellness apps; a large proportion of users quit within weeks (Baumel et al., 2019). Structured progress, with visible milestones and a clear "next step," has been shown to increase the number of completions compared to an open library, according to multiple studies (Baumel et al., 2019).

Simultaneously, the structured model will have its costs. This requires the client to curate and maintain the content, which provides less flexibility for users who wish to move freely. Urometoden tries to balance this by including a library framework that makes all content available for free exploration, alongside the structured course.

2.5 Relevant technology

REST API is the main architectural style for communication between the client and server in modern web applications (Fielding, 2000). Urometoden uses three endpoints that follow REST principles: resource-oriented URLs, HTTP methods that express intent, and JSON as the data exchange format. The architecture is chosen instead of complex alternatives like GraphQL because the model is simple and the client's needs are predictable.

The authentication patterns in modern web-apps are centralized around token-based solutions like OAuth 2.0 and JWT, often combined with session handling (Hardt, 2012).

Content delivery for audio files and media requires bandwidth, latency, and support for partial downloading (seeking). The standard HTTP Range Requests (RFC 7233) ensures it's possible to request parts of files, which is necessary for an audio player to use the material

without downloading the entire file each time (Fielding et al., 2014). Uromteden deploys this standard via Cloudflare R2, using the implemented streaming endpoint.

Large language models (LLMs) have, in a short time, become a relevant technology for health tools, both as a dialog mechanism and as a tool within the development process (De Choudhury et al., 2023). LLM was considered within Urometoden but was deliberately chosen not to be a core mechanism in the product, among other things, for privacy reasons and because the method's structure does not yet require generated responses. LLMs were still used as a tool in parts of the development; this is later discussed in chapter x.

3.0 Method

3.1 Research approach

The project follows an applied, constructive research approach; the goal is not to test a hypothesis in controlled environments but to build a concrete artifact for a real client (Hevner et al., 2004).
Qualitative and iterative methods are chosen instead of controlled experiments for many reasons. Firstly, the client's vision for the product was evolving throughout the project period. The requirements were not set by the time we started the project, but were adjusted through dialogue, culminating in a significant change of direction (pivot) along the way [3.3]. A controlled experiment presupposes stable variables and was thus not feasible. Secondly, the targeted audience comprises those with unrest and stress-related issues, who are difficult to recruit for formal studies within the framework of a bachelor's project.
In practice, the work with the client worked as a form of continuous demand valuation. The design decisions were presented, discussed, and agreed upon through weekly meetings, which provided an ongoing anchoring of the artifact in the client's professional understanding of Urometoden. This workflow reflects the principles of action research, in which scientists and practitioners collaborate to understand the problem and its solution.

3.2 Development method — agile/Scrum

The project has been carried out following agile development principles, with Scrum as an executive framework. Scrum is an iterative and incremental framework for production development, originally developed by Schwaber and Sutherland, in which work is organized into time-limited sprints with defined roles, ceremonies, and artifacts (Schwaber & Sutherland, 2020). The agile manifesto underlines that working software, collaboration with the customer, and the ability to respond to changes are more important than rigid plans and processes (*Principles behind the Agile Manifesto*, n.d.)
We used short iterations with weekly goals to structure the development. This approach enabled the group to handle changes, requirements, and priorities based on employer feedback, allowing it to deliver continuous functionality. Each week ended with a digital meeting with the employer, during which we discussed the changes made and the focus for the coming week.
The agile approach was not coincidental but was confirmed by the project's progress. The client's vision evolved significantly throughout the project, from an initially AI-based dialog to a structured digital journey without real-time AI interactions. This pivot, which is described further in 3.3, is in itself an argument for the agile approach: a waterfall project with locked requirements during the early phase would produce a solution that no longer answered the client's needs. The agile approach provided room to absorb this change without the project collapsing, as the decision horizon was never more than one sprint ahead.
Scrum's principle of delivering functional software frequently (Schwaber & Sutherland, 2020) was operationalized through weekly client demonstrations, in which the prototype was shown in its current state. This provides both the team and the client with material to discuss and reduces the risk of misunderstandings of requirements that accumulate over a longer period.

3.3 Data collection and client collaboration

The primary source for requirements and decision-making throughout the project was weekly meetings with the client. These meetings functioned as an informal yet systematic technique for collecting requirements, an approach similar to what the literature describes as an unstructured stakeholder interview, in which the conversation is controlled by context and needs rather than by a set questionnaire (Sommerville, 2016).

In practice, these meetings served three functions: (1) demonstration of the prototype's current state, (2) client feedback towards design, content, and course, and (3) priorities for the coming week's work. This cycle is similar to a Scrum sprint review but less formal, which suits the project's size and the client's role as a professional rather than a technical orderer.

The client's feedback affected the project on multiple levels. Fewer adjustments were made throughout the sprint, such as to the framework tone, content order, or wording in reflection tasks. The most comprehensive change was the pivot away from AI-based dialog as a core mechanism. The original concept was based on an LLM-guided conversation that guides the user through the Urometoden content. After multiple conversations, the decision was to move away from the LLM-based method, as it provided unpredictable responses in a vulnerable context, lacked control over therapeutic content, and privacy challenges related to third-party APIs. The decision to replace this with a curated, structured journey was made collaboratively and represents the most formative design decision in the project.

3.4 Tools and technologies

Tools	**Role**
Git/Github	Version control and collaboration
Trello	Task management and sprint planning
Figma	Design and framework prototypes
VS Code	Development environment
Cloudflare Pages	Hosting and automatic deployment
Cloudflare Wrangler	Local development environment for Workers, D1, and R2
npm/Vite	Package handling and system building
Discord	Fast communication
Betterstack	Application monitoring

3.5 Risk management

Risk management was handled throughout the project, with risk reviews connected to sprint planning. Shown in the table below:

Risk	**Probability**	**Consequence**	**Messures**
Unpredictability within LLM-generated answers: AI chat producing answers unfitting, wrong or potential harmful in a vulnerable context	High	High	Led directly to the pivot [3.3].
Dependencies on third-party APIs (originally OpenAI)-change in price, terms and availability.	Medium	High	Eliminated through the pivot from an LLM-based product.
Scope creep: Client's vision expanding faster than the group's capacity	Medium/Low	High	Short sprints with weekly priorities. Non-essential features explicitly deactivated.
Lack of content	Low	Medium	Data model designed to allow for client to add content without code changes.
Issues with audio streaming: Audio playback fails because of lacking Range Request support or network issues	Low	High	HTTP 206 partial content and ETag-caching implemented. Testing across devices and networks
Technical debt by fast prototyping: hard coded values, missing error handling, inconsistent condition management	Medium	Medium	Accepted as part of a prototype nature. Documented in the report [Limitations, 1.3] to ensure further development can address the issues
Losing access to	Low	High	Codebase inside Git

Cloudflare account			can be deployed inside an alternative infrastructure. No business-critical data inside the cloud
Sickness or a group member not available	Medium	Medium	Shared codebase, documented architecture, and frequent sharing of knowledge

4.0 Product Concept

4.1 Initial concept — AI chat

The original concept was an LLM-driven conversational bot used to guide users through Urometodens contents via a structured dialog. The thought was a language model, instructed to follow the methods' principles and work as a digital guidance, ask questions, respond to users' reflections, and adapt the conversation to the individual's needs. The model was intended to be anchored in the professional framework of the Uromethod through system instructions and contextual information, so that the answers remained within the method's limits.

This approach became the project's main course for much of the development process; approximately 70% of the overall development time was invested in it. The solution was built with a React/Vite-based chat client communicating via HTTPS with a FastAPI backend, which, again, is integrated with OpenAI's API (standard model gpt-o4-mini). Data was stored using PostgreSQL through parameterized `psycopg`, queries, conversation history, rolling summary, token budget per session, and authentication identities. Users log in via Google Sign-in or e-mail/password (`bcrypt` + e-mail verification), and session tokens are delivered as an HttpsOnly cookie.

Each message passed through a seven-step pipeline: security validation, idempotency check, token budgeting, theme classification, context window management, LLM call, and persistence. Prompt design was organized around configurable themes with dedicated system prompts, pacing rules, and security constraints. Guardrails included per-message token ceilings, injection heuristics, and generic error responses that never exposed internal details. Full pipeline specification is provided in Appendix B.

The implementation worked in practice. The system received user input, generated contextually relevant answers, and maintained the conversational flow over multiple messages. The API calls were stable, the response time was acceptable, and the framework provided a credible dialogue experience.

The problems were behavioral, non-technical. The language model could not reliably maintain Urometoden’s structure over a longer period.

Methodical operation: Tendency to provide users with generic mindfulness advice and empathy phrases instead of Urometoden’s specific progression and terminology. The longer the conversation lasted, the more the answers moved away from the method's core of the method.

Illusion of therapeutic relation: The dialog format created an implicit expectation that the system understood the user on a level it didn’t have the capabilities for.

4.2 The pivot — rationale and decision

The decision to leave the AI-based chat solution (ASHO) was made approximately in week 16 of the project. At this point, a significant portion of the development had already been invested in AI-based solutions. However, through testing and ongoing client feedback, it became clear that the approach did not align with Urometoden's requirements.

Client concerns. Bernard repeatedly stated that the model did not stay within the methods' framework. For a professional who has developed the method over a long period of time, it

was unacceptable that a digital tool could deliver content in the method's name without being accurate at all times.

Technical-methodological mismatch. Urometoden is built on a set progression: concepts are introduced in a specific order, tasks build on one another, and the tempo is deliberate. An LLM operates without memory for the progression unless it is explicitly introduced inside each conversation, and even then, it operates. The basic architecture in a language model (predict the next token based on probability) is not designed to maintain a pedagogical structure over weeks. The problem is not that the model is bad, but the task is not right for this tool.

Risk Assessment. As documented in risk management 3.5, unpredictability in a vulnerable context is identified as a high-probability, high-consequence risk. Incremental measures (better prompts, stronger guardrails, content filtration) were considered, but the conclusion was to reduce the probability of unfortunate answers without eliminating them. It was simply not enough for a product directed towards people with anxiety and unrest.

The pivot meant that the core mechanism was changed, from a generated dialog to curated content, but essential parts of the technical infrastructure were reused. The frontend framework (React/Vite), the hosting platform (Cloudflare Pages), and the serverless backend architecture (Workers Functions) were continued. The database was redesigned from supporting conversational logs to supporting content files per week (D1), and file storage (R2) was repurposed from eventual context storing to audio streaming. The pivot cost significant development time, but a moderate loss in infrastructure.

4.3 Revised concept — guided wellness journey

The revised ASHO concept is a structured digital platform delivering Urometoden’s content throughout an 8-week guided journey. In contrast to the original concept, in which the content was pre-made and curated by the client, nothing is generated in real time.

The journey is organized around weekly modules. Each week contains a curated selection of content elements from four types:

Audio files: guided meditations and somatic exercises delivered as audio playback.

Case-studies: recognizing stories, normalizing the user's experience. Shown as readable content inside a modal window.

Reflection tasks: Open, guided written exercises where the user formulates their own thoughts. Saved locally and privately.

Video contents: Pedagogical instructions for the week's theme and concepts.

The weeks are locked by progression: Only the active week and finished weeks are available, while the coming weeks are locked. This structure reflects Urometoden’s pedagogical design, in which early concepts (such as somatic consciousness) are prerequisites for later materials [2.4]. The user can view their progress through a visual timeline with status icons for each week (finished, active, locked) and a collected progress indicator.

In parallel with the structured journey, the application includes a library that provides access to all content across weeks, filtered by type. This accommodates users who wish to go back to earlier materials or explore freely. This balance between structured progression and free access is a deliberate design choice [2.4].

This concept is more accurate towards how Urometoden is used in a non-digital format, a set order, tempo, and content chosen and formulated by the client. The digital platform does not replace the client's role, but provides a scalable distribution where the method's integrity is maintained.

4.4 Target users and use cases

The primary user of Urometoden is adults searching for self-guidance for emotional regulation, typically unrest, stress, or anxiety during everyday activities, yet have no desire to seek traditional therapy.

Scenario 1- The weekly traveler. Paul (42) opens the app on Sunday evening and sees that week 2 is available. He starts with the audio exercise, reads the case study during the week, and writes in the reflection exercise before bedtime. On Friday, he checks the journey page to see his progress.

Scenario 2- The library user. Jeffrey (26) has completed the first four weeks but needs to revisit a specific audio exercise from week 2. He opens the Library, filters by “audio”, and plays it directly - without leaving his position in the structured journey.

4.5 Ethical considerations

ASHO operates in an environment where technology meets mental health- a framework that entails ethical responsibility beyond the technical. The exam requirements presuppose a discussion of ethical implications, and this section addresses the most central. Ethics is placed here, not only in the discussions chapter, because it was an active part of the design evaluation throughout the project.

Anonymity as a design principle. ASHO is designed to ensure the user never needs to reveal their real identity. All identifiers from login suppliers (Google and email) are hashed irreversibly via HMAC-SHA256 to a pseudonym `user_id` before being stored. The user can therefore authenticate themselves without the system storing who they are in a reversible form. Reflection texts are saved exclusively in the user’s browser (localStorage) and never leave the device. This approach is in line with the data minimization principle in Article 5(1)(c) of the GDPR, which establishes that personal data should only be gathered to the extent that they are necessary (GDPR, 2016). This is not only a juridical requirement but also a practical prerequisite for users working with emotionally sensitive material who dare to be open and honest about their reflections.

No clinical claims. ASHO is a content-delivery prototype, not a clinical tool. The app does not diagnose, treat, or provide medical advice. This distinction is more important than it may seem: every digital service that touches the subject of emotional health can be seen as something it’s not. The risk that users see the app as a replacement for professional help is real and must be handled proactively- through a clear disclaimer in the Welcome module section (De Choudhury et al., 2023). The risk was significantly higher in the original AI-chat concept: the dialog format created an implicit expectation that the system understood the user on a level it didn’t have the precondition for. The pivot to curated content significantly reduced the risk.

Responsibility for proximity to mental health. Even though ASHO isn't a treatment tool, it operates in a context where users may be vulnerable. This leads to three concrete responsibilities.

The first being the integrity of the content. Because all the content is curated by Bernard- not generated by an algorithm- the content is approved by a professional. This is an essential ethical advantage compared to the original concept and a direct consequence of the pivot. The responsibility for the content's quality and peculiarity lies with the method's author, not with the technology.

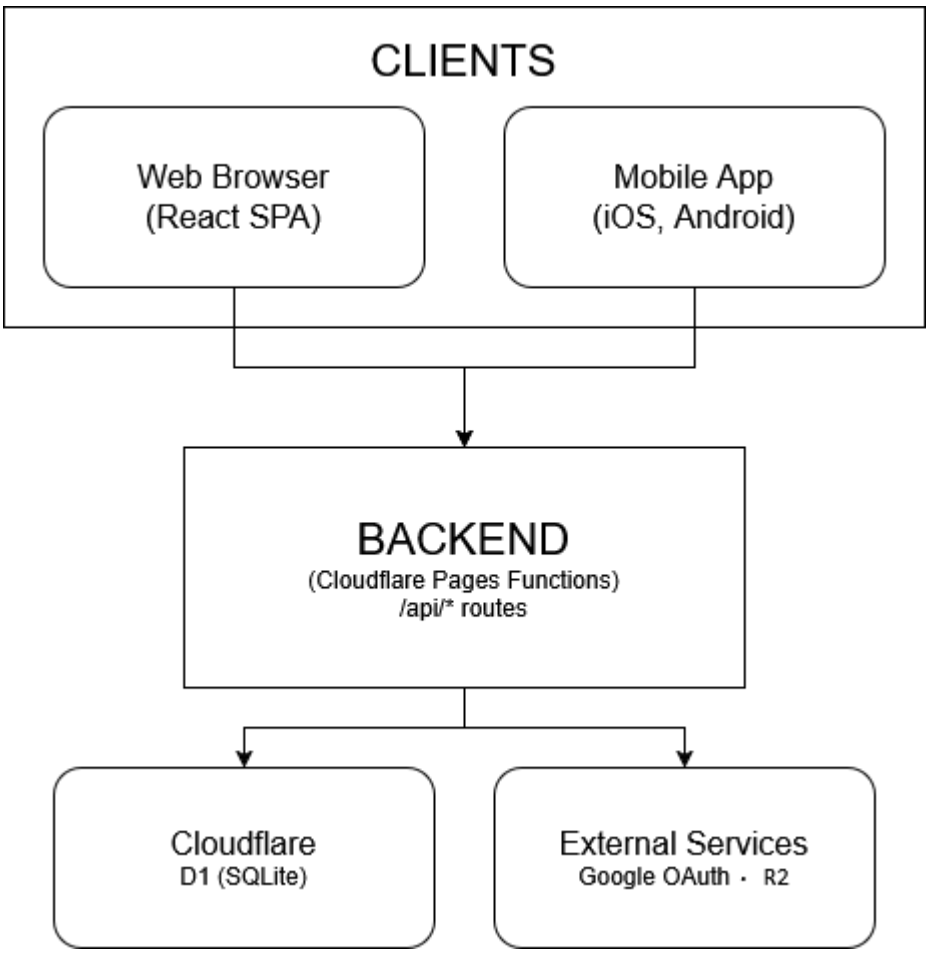
The second: Absence of an escalation mechanism. ASHO doesn't have any functionality to capture or respond to a user's crisis. There is no chat function, alert system, or connection to health personnel. In a product version, it should be considered adding visible information about external services- not as a function in the app, but as available information for users needing something the app can't provide.

The third: reflection-data and self-exposure. The reflection exercises invite the user to write freely about their own experiences with unrest and anxiety. The act may create a form of self-exposure that should be treated with respect, even though it's stored locally. The framework should clearly communicate that the texts are private, not shared, and that the user has full control over them, including the ability to delete them.

The pivot is an ethical decision. In summary, the pivot from the AI-chat to curated content is not only the most important technical decision within the project- it's also the most important ethically. It went from technical elegance to content control, prioritizing the user's security over functional ambition. This priority permeates the design: anonymity before personalization, local storage before the cloud, curated content before generated content.

5.0 System overview

5.1 Architecture overview



Urometoden uses a classic client-server architecture where presentation logic and user framework is separated from data access and business logic. This separation is an established principle in modern web applications (Fielding, 2000), providing two concrete advantages: frontend can be developed and tested independently to the backend (crucial during the pivot), and the client code can be distributed via a global CDN without exposing the database or storage infrastructure.

The architecture is built on 4 layers (diagram above)

The Client layer is a React 18.3 application built with Vite. The app handles all presentation logic, navigation, and local state-handling.

The Hosting layer is Cloudflare Pages, which delivers the frontend application via a global CDN and automatically deploys when something is pushed to the master branch. This provides low latency for end-users and eliminates the manual deployment process.

The API layer is built with Cloudflare Workers Functions, serverless functions that run on Cloudflare's edge network. Three REST endpoints expose content data to the client. Serverless architecture was chosen ahead of a traditional application server because traffic patterns are unpredictable (prototype phase) and costs are scalable.

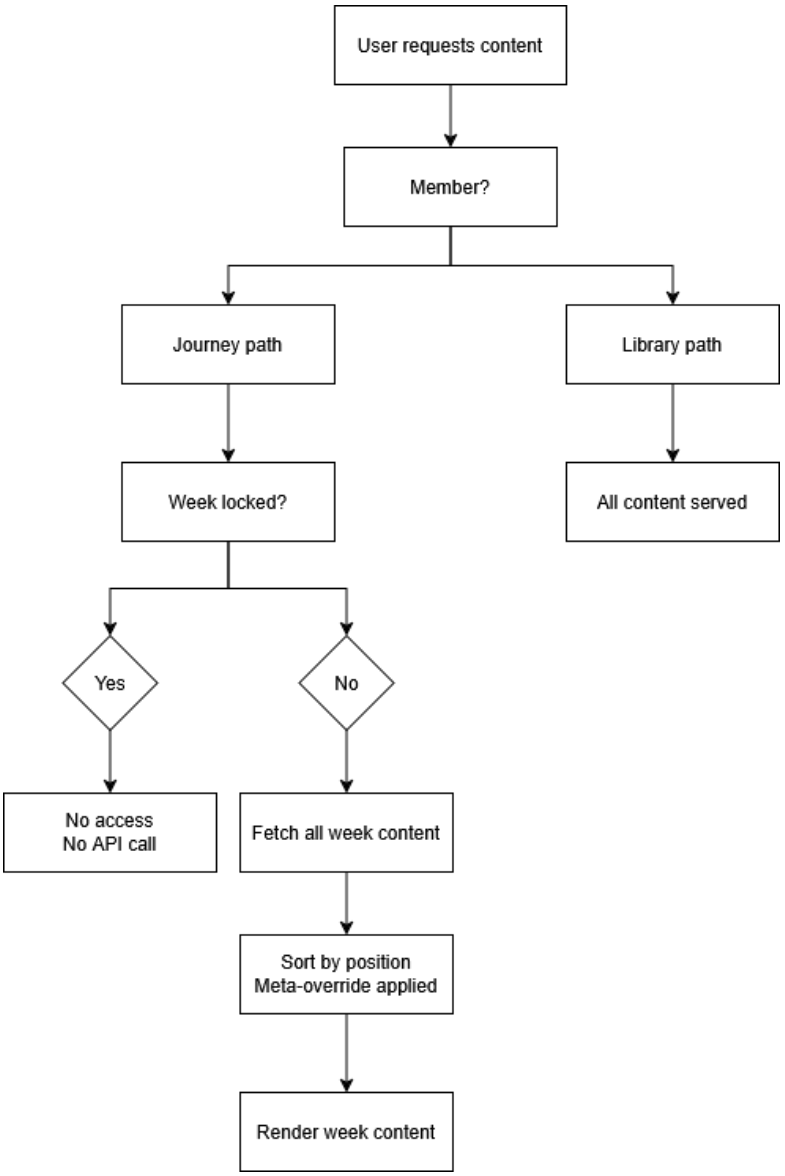
The Data-layer is divided into two: Cloudflare D1 (administered SQLite) for structured content data and metadata, and Cloudflare R2 (S3-compatible object storage) of audio files.

This separation reflects that structured data and binary media files have different access and delivery patterns.

Autentisering is implemented with three identity suppliers (Google and email/password) that converge on a common session layer, described in detail in 5.4.

The subscription handling is prepared in the framework via a trial-period badge, but the backend logic for access control based on subscription status is not implemented in the prototype.

5.2 Content delivery model



Urometoden's contents are delivered through two parallel paths, as shown in the diagram above.

The structured journey is the main path. The user navigates to a specific week, while the client checks the weekly status (finished, active, or locked). A request to `GET /api/{weekId}/content` is sent if the week is available, which performs a JOIN between the tables content_items and week_content. The result is a sorted list of content elements

for the relevant week, where position determines display order, and weekly-specific metadata can override generic metadata via a meta-override mechanism.
The library is the alternate path. The user receives all content via GET/api/content without weekly filtering. The user can filter content type (audio, case, reflection, video) via a query parameter. This path includes no unlocking logic, and all content is visible.
Both paths end in the same client-side display logic, where content elements are routed to the correct component based on type: audio exercises to AudioPlayer, case-studies to CaseModal, reflection tasks to ReflectionModal, and videos to the relevant video display.
There is an extra delivery for audio content through `GET/audio/{filename}`, which streams the file directly from R2. This endpoint supports HTTP 206 partial content for seeking, ETag headers for caching, and returns a Cache-Control header with an hourly lifespan.

5.3 Backend modules

The backend layer is compact, composed of three serverless functions, each with a clear responsibility.
Content list (`GET /api/content`) returns all content elements from the `content_items` table as a JSON array. It supports optional filtration via query-parameter type. This function is stateless; it reads from the database and returns the result without any transformation.
Weekly content (`Get /api/weeks/{weekId}/content`) is the most composite function. It executes a JOIN between `content_items` and `week_content`, sorts the result by position, and applies meta-override-logic: if week-specific metadata is set, it overrides generic metadata. This mechanism allows the same content element to be reused across multiple weeks with different contextual metadata.
Audiostreaming (`GET /audio/{filename}`) handles delivery of binary audio files from the R2-storing. The function implements HTTP Range Requests (RFC 7233) to support seeking within the audio player, returns 206 Partial Content in response to range requests, and sets ETag- and Cache-Control-headers for effective caching. Non-existing files return as 404.
Database schema. The data model consists of two tables. `content_items` saves all pedagogical content, with fields for type, title, metadata, R2-key (for audio/video), abstract, body text (for case studies), and reflection instructions (for reflection exercises). `week_content` is a connecting table that ties content to weeks with position and optional metadata-override.

5.4 Authentication and sessions

ASHO supports three identity suppliers that converge on a single common session layer. Google Sign-In verifies its ID token against Google's public key, validating the issuer, the expiration point, and the subject, and hashes the subject irreversibly to a pseudonymous `user_id` via HMAC-SHA256. E-mail/password is handled by its own service using Argon2id hashing (bcrypt as a fallback), e-mail normalization, and a complete registration, verification, and reset flow.
Sessions are opaque UUID's stored in a sessions table with expires_at set to `NOW() + SESSION_TTL_DAYS`. Each call to `get_session_principal` cleans expired sessions and

returns the `user_id` and an `is_admin` flag for role separation. Log out deletes the session row immediately. The client sends a session token as `Authorization: Bearer` header instead of a cookie, which eliminates CSRF risk against the API endpoints.
Rate limiting is implemented with two layers: one in-memory deque per IP address for the Google flow, and one token-bucket limiter per combination (flow, IP) and (flow, identifier) for registration, login, reset, and verification.
Common for all three suppliers is that the original identifier is never stored as a primary key, they are all hashed to a pseudonym `user_id`, which ensures that even with a complete database leak, the user identity can't be reversed without access to the HMAC-secret. The detailed review of the security architecture continues in Chapter 8.

5.5 External integrations

The Cloudflare ecosystem is the only active external dependency in the current product. Pages, Workers, D1, and R2 constitute the entire infrastructure. Every service is administered via Cloudflare Wrangler CLI and deployed automatically. This consolidation, everything with one supplier, simple drifting, and reduced integration risk, but creates a platform dependency that should be looked at in further development [3.5].
Google Sign-In is an active integration carried over from the AI chat phase and provides two of the three authentication paths described in section 5.4. A third path (email/password) was added during the pivot phase.
OpenAI API was the central external dependency during the AI-chat phase, where GPT-o4-mini ran the dialog mechanism. This feature is no longer active after the pivot. The API key and its configuration still exist in the codebase, but are not used by any user-facing functionality. The decision to keep the integration code is to preserve the opportunity to re-integrate a limited AI in future versions [look 11.5] and to serve as documentation of the original architecture.
BetterStack is used for application monitoring and provides insight into uptime and error rates. This integration is lightweight and operates independently from the codebase.

6.0 Design

6.1 Design principles

The design is guided by three principles that directly address the targeted audience and user context described in 2.3 and 4.4.

Calmness before stimulation. Users usually enter the application in a state of unrest; the design should not elevate this feeling. This means a lowered, warm color palette, absence of aggressive elements such as countdowns, gamification points, or notifications, and spacious typography with good contrast. The choice of forest green (#3D6B5A) as the primary color and antique gold (#C4A46B) as the secondary color is chosen as earth-tones associated with nature, safety, and stability (Eriksen, 2024). This stands out in contrast to the strong blue- and purple-tones that dominate other wellness apps such as Headspace and Calm.

Structural clarity. The framework should make it clear what the user can do at any given moment, without being overwhelmed by choices. This is operationalized through a clear visual hierarchy between the week's content (primary), navigation (secondary), and statistics (tertiary). Color coding of contents, blue for sound, purple for case, green for reflection, orange for video, provides immediate recognition without the user needing to read the etiquettes. This convention is consistent throughout the entire application: content cards, library filters, and week pages use the same color system.

Minimal friction. Each interaction requires the least amount of steps. The audio player is always available at the top of the weekly page; the reflection modal opens with one click; and case studies are shown inline without navigating away from the context.

6.2 User journey

The user journey within ASHO follows parallel paths that meet in the same content, as described in 5.2. Here, we describe the user's experience.

First time visiting. The user arrives on the dashboard (home page) and is greeted by a welcome modal that briefly explains what the application offers. The modal is shown only once because it is stored in localStorage (can be accessed manually later). After the modal is closed, the user will receive a personal welcome based on the time (morning, midday, or evening), a progress card showing week 1 as active, and a daily tip that rotates based on the calendar date.

Weekly cycle. The user navigates to the active week via the sidebar or the progress card. The week page presents all available content for the week in a grid: audio exercises at the top, with an integrated audio player, followed by a content card for case studies, reflection exercises, and video. The about-card at the bottom describes this week's theme. The user works through the content in their own order and at their own pace.

Progression experience. The journey-page (JourneyPage) provides the user with a bird's-eye view: a visual timeline over all eight weeks with connection lines, coloured status indicators (green for finished, yellow for active, and grey for locked), and an overall progress indicator. This page is designed to give the user a sense of accomplishment and orientation. The user can easily track their progress.

6.3 Wireframes and prototyping

The design process evolved from a low-fidelity Figma wireframe to interactive React prototypes. Early concepts explored an AI-oriented chat structure; after the pivot, the design shifted to an 8-week journey requiring significant rework of navigation, onboarding, and content presentation. A mobile-first redesign adapted the experience using slider-based onboarding and compact layouts inspired by modern wellness applications, while preserving Urometoden's visual identity.

Component based on prototyping. The interface was developed component by component, where individual elements were designed and tested independently before being integrated into a larger page structure. Reusable components were created for navigation, audio playback, reflection tasks, and case studies. This modular approach made it easier to redesign to adjust the interface throughout the project.

Iteration with the client. The prototype was demonstrated to the client weekly, as described in section 3.3. Feedback, regarding the visual tone, content structure, navigation, and the user experience.

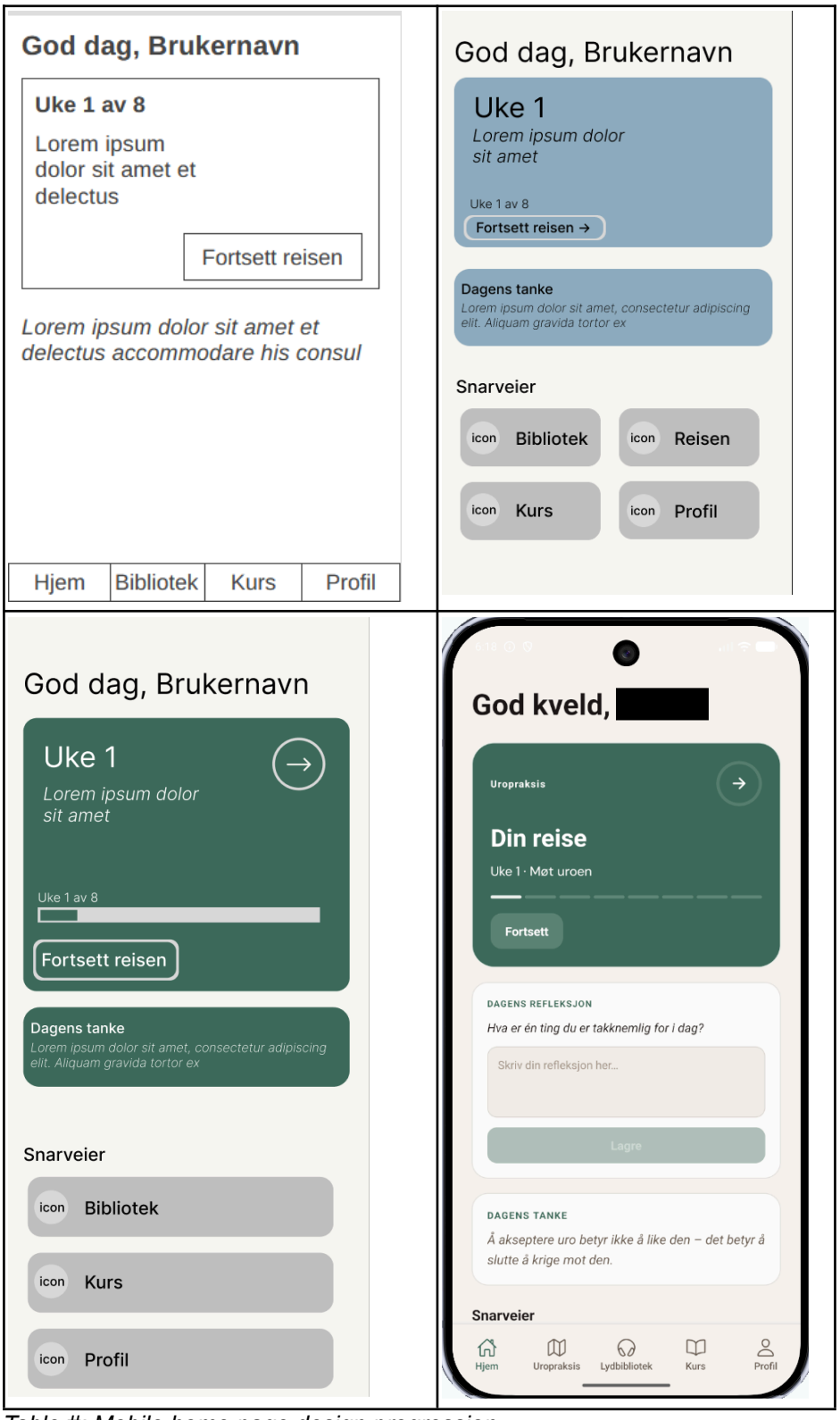


Table #: Mobile home page design progression.

6.4 Visual design decisions

Colour pallet. The design system is built around a warm earth-tone palette. The background is warm beige tones in day mode and a neutral, low grey tone in night mode. This duality is implemented using CSS variables in a central theme file (theme.css), allowing the entire color profile to be changed by modifying a single variable group.
The content types have dedicated signal colors that break with the otherwise muted palette: blue for sound, purple for case, green for reflection, and orange for video. These colors are chosen to be distinctive from one another and from the background tone, ensuring each type can be easily identified.
Typography. Three font families are used, each with a distinctive role. DM Serif Display is used for headlines and provides a classic, confidence-inspiring character. DM Sans is used for body text and framework elements, a neutral sans-serif that is easily read on screen. Cookies are used sparingly as a decorative font to add personality without compromising readability.
Layout. The main layout is a CSS Grid with three columns: a foldable sidebar (48px collapsed, 230px expanded), the main content area, and a right panel. The sidebar shows the user's journey with a progress indicator and weekly list, while the right panel shows statistics, next week card, and testimonials. This three-column structure makes a dashboard-character which includes all relevant information without requiring navigation.
Theme. Day and dark modes are controlled by CSS variables and toggled via a sun/moon toggle in the navigation bar. The app respects system preferences at first visit (via `prefers-color-scheme` media query) and stores the user's choice in localStorage. Theme change occurs through smooth transitions to avoid abrupt visual changes.
Animation. The full-screen audio player features an animated wave background using HTML5 Canvas, designed to create a calm, meditative atmosphere without distraction.

6.5 Accessibility considerations

Accessibility is especially relevant for Urometoden as the targeted audience includes people in vulnerable conditions where cognitive overload and sensory sensitivity can be elevated (Initiative (WAI), 2024; *W3C*, 2025).
Implemented: The color contrasts within the design are chosen with readability in mind, the warm beige background provides sufficient contrast against dark text in day mode, and the neutral grey tones in night mode provide the equivalent contrast for bright text. The content types signal that colors are never used alone as information carriers; they are always supplemented with text, etiquette, and type tags on the content cards.
The audio player supports keyboard navigation with shortcuts: space for play/pause and the arrow keys for seeking. Volume control is available via the framework, and time indication provides ongoing orientation about the position inside the audio file.
The theme change (day/night mode) gives users control over visual comfort, which is especially useful for users with photosensitivity or who use the application in dark environments.

7.0 Frontend & Mobile Application

The platform is delivered through two clients: a React SPA for web and a React Native Expo application for mobile, both consuming the same Cloudflare Pages Functions API and sharing the same visual identity. The chapters below cover each client's structure, key feature implementations, and the integration patterns that connect them to the backend.

7.1 Web client — structure and routing

The web client is located under frontend/src/ and follows a flat, page-oriented structure. There is no feature-based folder hierarchy — pages are colocated in src/pages/, reusable UI components in src/components/, and shared utilities and hooks directly under src/. Styles are written in CSS Modules, with one .module.css file per component. Global design tokens — colors, spacing, typography, and dark mode overrides — are defined as CSS custom properties in src/theme.css and applied to the document root. Routing is handled by React Router v7, configured declaratively in App.jsx. A <BrowserRouter> wraps the application, and routes are defined with <Routes> and <Route> elements. The route table is as follows:

Path	Component	Access
/	DashboardPage	Members
/praksis	JourneyPage	Members
/praksis/uke/:weekid	WeekPage	Members
/bibliotek	BibliotekPage	Members
/kurs	KursPage	Members
/hjelp	HelpPage	Public
/admin	AdminPage	Admin
/onboarding	OnboardingPage	Public

7.2 Weekly journey implementation

Route protection is handled at the component level rather than through wrapper routes. On mount, each protected page reads the user hint from localStorage and verifies membership status. Unauthenticated users are redirected to /onboarding via useNavigate. The application shell renders a persistent three-column grid layout regardless of the active route. The left column contains a collapsible sidebar with week-level navigation; the center column renders the active route; and the right column displays a stats and progress panel. This

layout collapses to a single column on narrower viewports via CSS media queries. Dark mode is toggled by adding or removing a dark class on the <html> element. All color values are defined as CSS custom properties with light defaults and HTML.dark overrides, meaning no JavaScript re-render is required when the theme changes. The user's preference is persisted in localStorage.

7.3 Library view

The library gives members access to the full content catalog independent of their weekly progression. Unlike the journey, no content in the library is gated by week status — all published items are visible to any active member. Content is fetched from /api/content, which returns a flat list of items. Each item carries a type field (audio, reflect, case, or video), a title, an abstract, a meta string encoding duration, an optional r2_key referencing the audio file in Cloudflare R2, and a weeks array indicating which weeks the item belongs to. The client groups items by type and renders each group as a labeled section. On the web, BibliotekPage supports type-based filtering via a query parameter. Selecting a filter re-fetches the list from/api/content?type={filter} and re-renders it. Opening an item launches a modal appropriate to its type — a full-screen audio player for audio content, and a reading modal for reflections and case studies. Playback position and listen time are written back to the API on pause and on unmount, allowing the user to resume from where they left off. On mobile, app/(tabs)/lydbibliotek.tsx follows the same pattern. Content and progress are fetched in parallel on focus via useFocusEffect. Each item is rendered as a ContentCard displaying the title, type, duration parsed from the meta field, and a progress bar derived from position_seconds / duration. Tapping a card opens a ContentModal. Audio playback is managed by a shared PlayerProvider context that persists across tab navigation, keeping the mini-player active while the user browses other screens.

7.4 Mobile client

The mobile client is located under app/ and is written in TypeScript using React Native with Expo SDK 54. File-based routing is provided by Expo Router, which maps the directory structure under app/app/ directly to the navigation tree. The root layout at app/app/_layout.tsx wraps the application in four context providers — AuthProvider, ThemeProvider, PlayerProvider, and an auth guard that redirects unauthenticated users to /onboarding — before rendering the tab navigator. The main interface is a five-tab bottom navigation defined in app/app/(tabs)/_layout.tsx:

Outside the tab group, dynamic and modal screens are registered as stack routes: app/uke/[id].tsx for individual week content, app/auth/signin.tsx for authentication, app/onboarding/index.tsx for the onboarding flow, and app/paywall/index.tsx for the membership gate. Typography uses DM Sans, loaded via @expo-google-fonts/dm-sans, in four weights. Because iOS does not apply fontWeight to custom fonts without an explicit family name, a custom Text component is at app/components/ui/Text.tsx wraps React Native's Text, reads the fontWeight from the flattened style, and maps it to the correct named font family before rendering. All screens import this wrapper rather than the native component. The color system is defined in app/constants/colors.ts and exposes separate LightColors and DarkColors objects. The active set is selected by ThemeContext based on the system color scheme, accessed via useTheme() throughout the component tree. Dark mode follows the OS setting and requires no explicit toggle from the user.

7.5 API integration and state management

Neither client uses a global state management library. State is kept local to the component or screen that owns it, with custom hooks encapsulating fetch logic and exposing a stable interface to the UI. Web: The web client calls the API directly using the native fetch API within page components and hooks. There is no shared HTTP client or request interceptor. Authentication relies on a JWT validated server-side on each request; the client holds a lightweight user hint object in localStorage containing display name, membership status, and admin flag, used only to drive UI decisions without an additional round-trip. Custom hooks such as useUserProgress and useWeekProgress encapsulate the fetch-and-setState pattern for the data they manage, and are called at the top of the relevant page component. Mobile The mobile client wraps all API calls through a single apiFetch utility in app/lib/api.ts. The function accepts a path, optional fetch init options, and the user's JWT, and constructs the full request against the configured base URL (EXPO_PUBLIC_API_URL, falling back to the production domain). The Authorization header is injected automatically. The JWT itself is stored in expo-secure-store under the key uro_token and read from AuthContext on each call. On app launch, the token's exp claim is decoded locally and checked against the current time; an expired token clears the auth state and redirects to the sign-in screen before any API request is made. Shared patterns: both clients follow the same conventions: data is fetched on mount or focus, loading and error states are managed locally with useState, and no response is cached beyond the screen's lifetime. Write operations — saving a reflection, marking content complete, recording playback position — are fire-and-forget POST or PATCH requests with optimistic UI updates where latency would otherwise be noticeable. Neither client implements retry logic; failures surface as silent no-ops or inline error messages depending on the operation.

8.0 Security

8.1 Authentication and access control

ASHO supports tree identity providers, with each provider stored for a common user and session-layer.

Google Sign-in verifies the ID token against Google's public key via `google.oauth2.id_token.verify_oauth2_token`, where `GOOGLE_CLIENT_ID` works as an audience parameter. Issuer (accounts.google.com), expiration time, and subject are explicitly validated before HMAC-hashing to a pseudonymous `user_id`. Google implements double-submit patterns using its own CSRF defense: g_csrf_token in the request body, which is compared against an equivalent cookie value before credentials are processed.

Sessions are opaque UUIDs stored in a session table, with expires_at set to `NOW() + SESSION_TTL_DAYS`. Each call to `get_session_principal` cleans out expired sessions and returns the `user_id` and an `is_admin` flag for role separation. Logout deletes the session row immediately. The client sends the session token as `Authorization: Bearer` header, not as cookies; this eliminates CSRF risk against the API endpoints.

Access control is enforced at an endpoint level. The Chat-endpoint requires a valid bearer token, looks up `user_id` from the session, and denies requests to conversations owned by other users with HTTP 403. This protects against Insecure Direct Object References (IDOR), one of the most common vulnerabilities in the OWASP Top Ten (OWASP Top 10:2025, n.d.). The role separation between users and administrators is supported through the `is_admin` flag inside the session principles.

Subscription gating is configured within the framework using a trial-period badge (“7 days for free”). The backend logic for subscription status, like the access control layer, includes conditions: trial, active, expired, and canceled.

Rate limitation is implemented in two layers. The Google flow uses an in-memory deque per IP address. Registration, login, reset, and verification are protected by a token-based limiter that limits both per combination (flow, IP) and (flow, identifier). This separated approach stops both distributed attacks from many IPs against an account and focused attacks from a single IP against multiple accounts.

8.2 Data protection and privacy

ASHO’s approach to data security combines data minimization with pseudonymization: the system collects only the minimum necessary data, and what's collected is not directly linked to the user's real identity.

Pseudonymisation of identity: Google's subject identifier is not saved as primary key. All the identifiers are used as input to a HMAC-SHA256 with a separate secret (`AUTH_SUB_ASHSECRET/GOOGLE_SUB_HASH_SECRET`), and the resulting hash becomes the user's `user_id` reversed to the original identifier without access to the HMAC-secret. Email identities are stored in a separate table (`user_email_identities`),
separated from the user's data – a conscious design decision that limits exposure by possible compromise of individual components.

Password storage: Passwords are never stored in clear text. `_PasswordEngine` hashes with Argon2id as priority, recommended by OWASP as the strongest available algorithm for password storing (Password Storage - OWASP Cheat Sheet Series, n.d.). BCrypt with random salt is used as a fallback. `verify_password` automatically selects an algorithm based on the hash-string prefix ($argon2 or $2), enabling seamless migration between algorithms without users needing to reset their passwords. This flexibility aligns with best practices for cryptographic agility (*Password Storage - OWASP Cheat Sheet Series*, n.d.).
Cookies for nonce are set with `HttpOnly`, `Secure` (requires HTTPS), and `SameSite=lax`, with a short timeline (`AUTH_OAUTH_COOKIE_TTL_SECONDS`, default 15 minutes). The cookies are deleted immediately after the callback, minimizing the window during which they can be abused.
Error handling and information leakage: The API router returns generic errors ("Internal server error") instead of stack traces, preventing internal implementation details from being exposed to attackers. The forgot-password-endpoints answers identically independently of if the account exists, which prevents user enumeration (*Authentication - OWASP Cheat Sheet Series*, n.d.). The security code prevents the explicit echoing of unreliable input in error details, a practice that helps protect against reflected information leakage.
Security log: Rejected messages are stored with only a 500-signs preview and a `rejection_type` classification, not the entire message content. This approach balances the need to discover and analyze attacks against the principle to minimize storing potential sensitive content.
GDPR: The Personal Data Protection Regulation (GDPR) is incorporated by Norwegian law through The Personal Data Act of 2018, with Datatilsynet as the supervisory authority (*Personopplysningsloven*, 2018). Multiple provisions are directly relevant. Article 5(1)(c) establishes data minimization; ASHO's pseudonymization architecture is a strong implementation of this principle. Article 25 requires built-in privacy (data protection by design and by default). HMAC-based identity handling is an example of this. Article 9 provides protection to especially health-related information, reflection texts where the user writes about anxiety or unrest can potentially qualify as such. Reflections stored locally in localStorage fall outside the processing responsibility; if a future version introduces server-side storage of reflections, this triggers requirements for explicit consent, an impact assessment (DPIA), and potentially a data processing agreement with the cloud infrastructure provider (*Datatilsynet*, n.d.).

8.3 Input validation and injection mitigation

Input validation within Urometoden is implemented as a five-layer pipeline: schema validation (Pydantic), Unicode normalization (NFC), character allowlist (ASCII, Norwegian characters, limited punctuation), token cap (512 per message), and prompt-injection heuristics. Idempotency on (conversation_id, message_id) via SHA-256 hash prevents duplicate processing. SQL injection is prevented throughout by using parameterized queries (SQL Injection Prevention, (*SQL Injection Prevention - OWASP Cheat Sheet Series*, n.d.). React 18 provides baseline XSS protection by escaping all JSX values (Meta Open Source, n.d.). Detailed per-layer specification is in Appendix C.

8.4 CORS and transport security

CORS is configured centrally within the application's startup file. `allow_origins` are read from the environment variable `ALLOWED_ORIGINS` (a comma separated list); it will fallback to wildcard (`["*"]`) if not set. `allow_credentials` is set to `True` until all methods and headers are allowed. A separate origin-allowlist is used for redirect-URLs: `_pick_redirect_url` compares `return_to` / `Origin` against the allowlist and removes any unknown values, helping prevent open-redirect attacks.

Cookies are set as standard with `Secure=True` (requires HTTPS), `SameSite=lax`, and `HttpOnly=True`, which provides solid defense against cookie theft via XSS and CSRF attacks. Inside the development environment, it requires HTTPS transport for the cookie flags to be respected by the browser.

CSRF protection is handled through two mechanisms. The double-submit cookie pattern is used in Google authentication, where the g_csrf_token in the body is compared with the corresponding cookie value. Bearer-token inside the `Authorization` header is used for API calls, instead of cookie-based authentication, which makes CSRF irrelevant for these endpoints, recommended by OWASP CSRF Prevention Cheat Sheet (*Cross-Site Request Forgery Prevention - OWASP Cheat Sheet Series*, n.d.).

8.5 Token hashing and integrity

ASHO's token-architecture is built upon a throughout principle: secrets are never stored in a reversible form inside the database.

Provider-identity (Google sub and email) hashes with HMAC-SHA256 and a dedicated secret (AUTH_SUB_HASH_SECRET) to generate a pseudonymous `user_id`. A provider prefix (`email_`or Google) separates domains to ensure hash collisions between different services can't collapse identities, a subtle but important decision that prevents a malicious actor from creating an identity with one provider that clashes with another provider's user.

Verification and reset tokens are generated with `secret.token_urlsafe(32)`, but only the HMAC hash (with `AUTH_TOKEN_HASH_SECRET`) is stored within the database as `verify_token_hash or reset_token_hash`. The client must present the clear-text token and the server-side hashes, and have them compared. This approach implies that a full database leak doesn't directly make the tokens usable; the attacker would also need access to the HMAC secret. The token lifespan is approximately 30 minutes (`AUTH_EMAIL_VERIFY_TOKEN_TTL_MINUTES`, `AUTH_RESET_TOKEN_TTL_MINUTES`), and SQL queries require `expires_at ⩾ TO_TIMESTAMP(now)`. This ensures expired tokens are rejected even though they are not cleaned from the database.

.**Password-hash integrity** is taken care of through Argon2id, and bcrypt both carry their parameter inside the hash string. The verification function chooses algorithms based on prefixes (`$argon2`, `$2`) and rejects hash-strings with unknown prefixes, preventing downgrade attacks in which the weaker algorithm is forced through.

An idempotency hash over {conversation_id, message_id} using SHA-256 ensures that a reused message_id with changed content is rejected with a 409 Conflict, while identical requests are recognized as duplicates and not processed again (Krawczyk et al., 1997).

9.0 Results

This section presents the implemented features of the delivered prototype, documents the system in its current state, and includes an evaluation from the client on the outcome of the project.

9.1 Implemented features

The following table gives an overview of some of the major features delivered as part of this project, with a brief description of each and their current implementation status.

Feature	Description	Implementation Status
Admin control panel.	An admin control panel displaying various site and user statistics, as well as tools to create and manage content hosted on the web page/mobile app.	Implemented.
8-week course architecture.	Full front- and backend architecture for an 8-week course program, including design, backend functionality, user access, database, and admin CRUD options.	Implemented.
Full content library.	A library serving all available content for users to browse at will. This includes a Cloudflare R2 bucket hosting media files, linked to a Cloudflare D1 database.	Implemented.
Live events.	User-facing live event list, including physical events and online meetings, powered by a live-counting backend fetching from the database to sort events by recency and automatically archive events past the current date.	Implemented.
Authentication & membership	Email and Google OAuth sign-in on web and mobile, issuing JWTs validated on every API request. Membership status is checked on the server to gate member-only content. Tokens are stored in device's secure storage on mobile.	Implemented. Memberships implemented, but admin-managed. Billing out of scope for the project.
Audio playback.	Full-screen audio player streaming media from Cloudflare R2 via signed URLs, with a	Implemented.

	persistent mini-player, scrubbing controls, and resume-from-position support on web and mobile.	
User progress tracking.	Per-user D1 records tracking week start/completion timestamps and per-content listen time and playback position. Gates sequential week access and surfaces completion stats on the dashboard.	Implemented.

9.2 System in operation

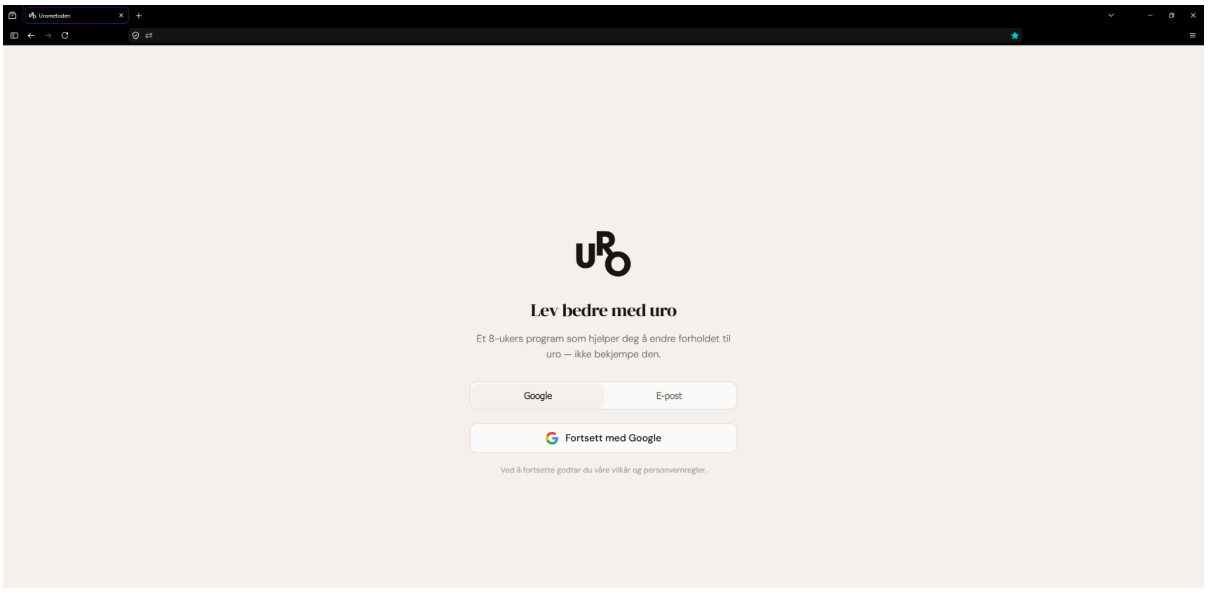


Figure X: Web login page

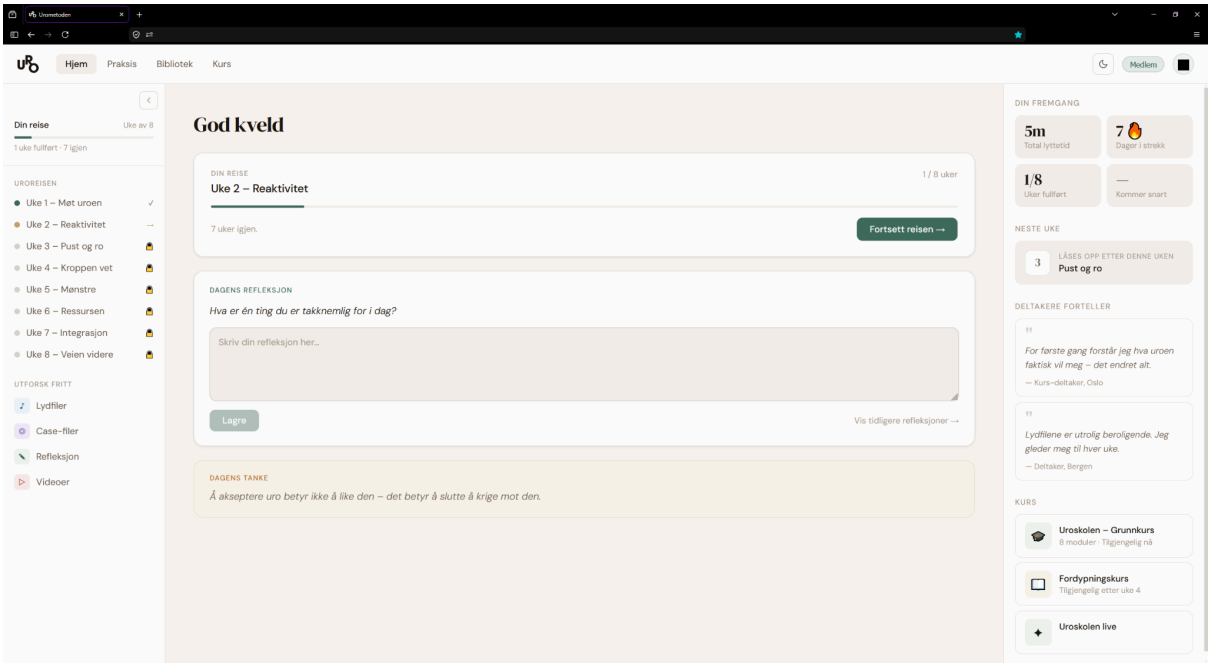


Figure Y: Web home page

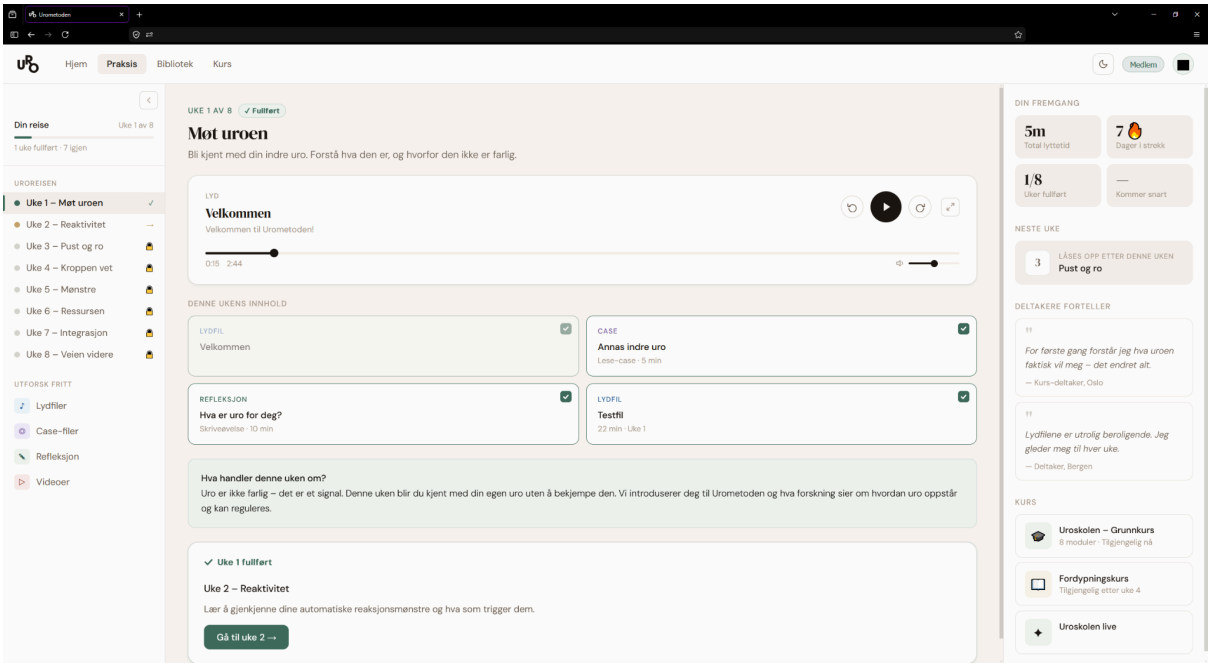


Figure Ø: Web week 1 page

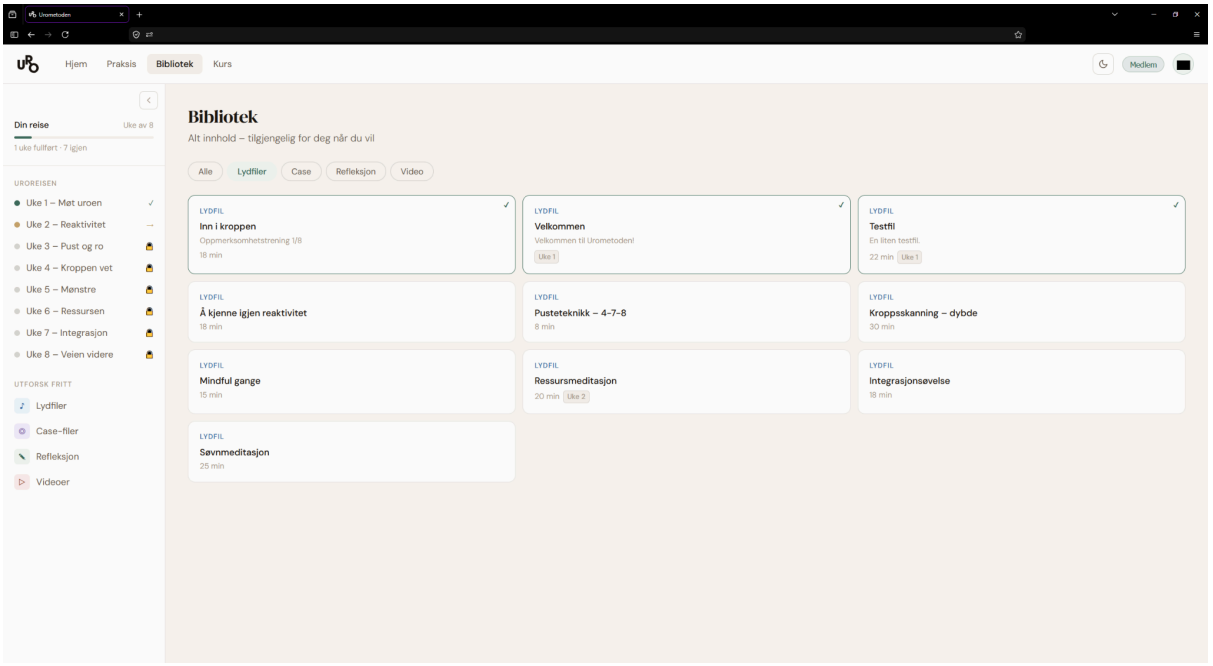


Figure Z: Web library page

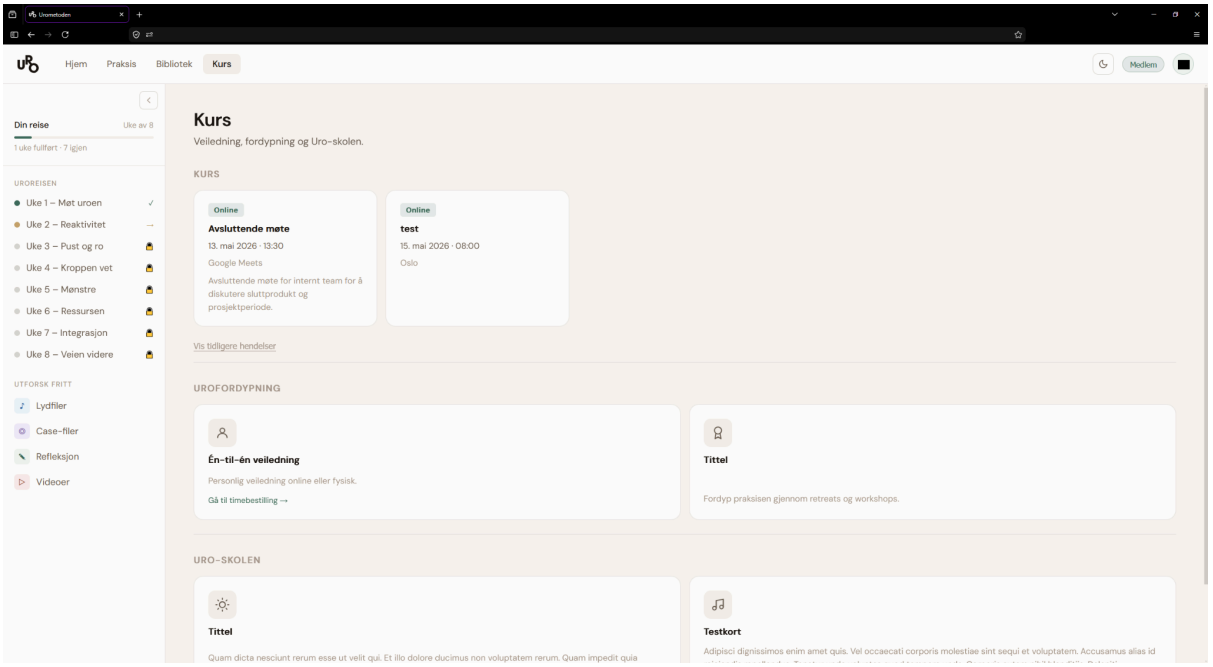


Figure Æ: Web courses page

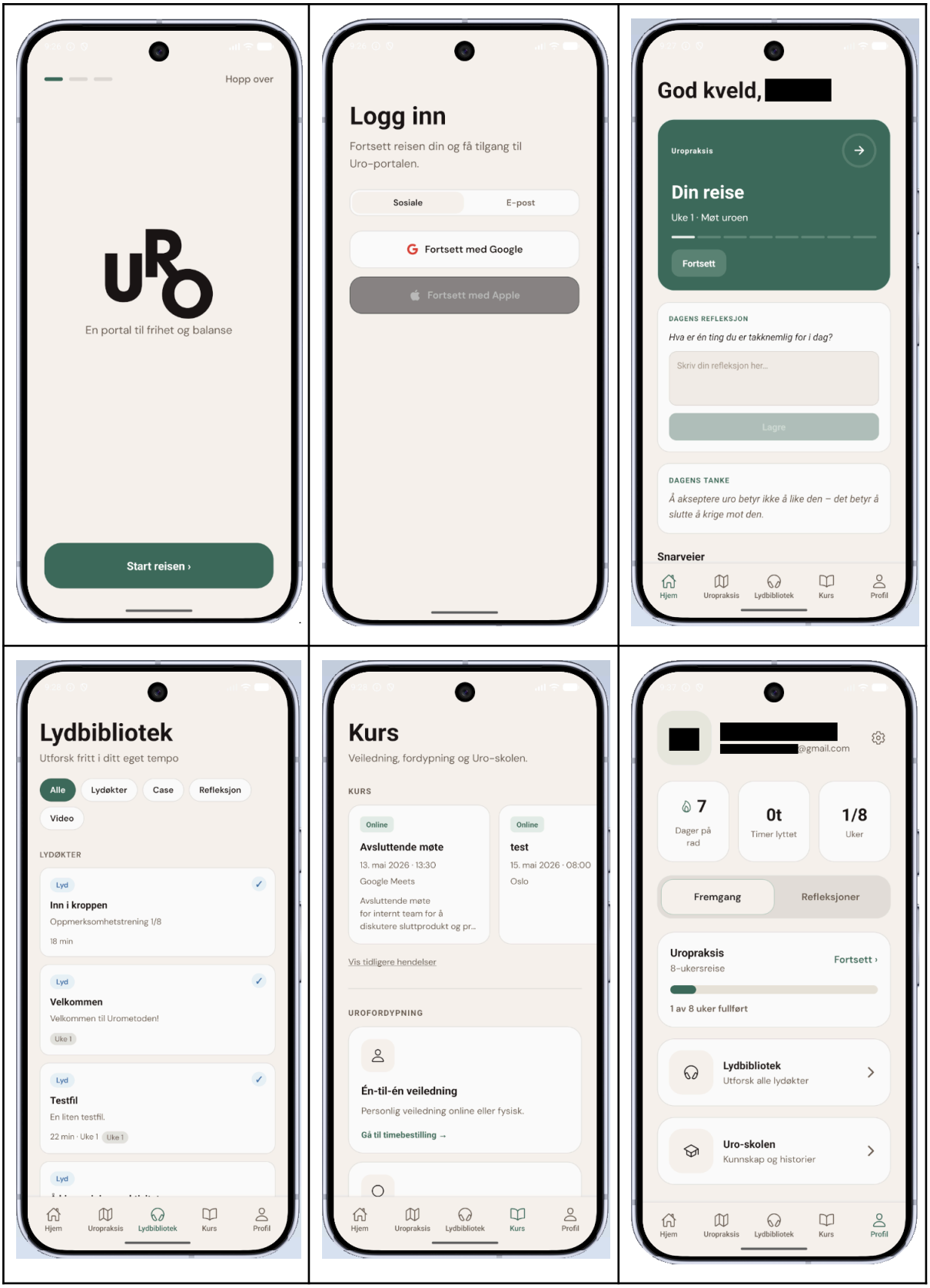


Table X: Mobile app pages overview

9.3 Client evaluation

The final client meeting with Bernard H. Bøhmer served as the primary evaluation of the delivered prototype. As both the creator of Urometoden and the project's principal stakeholder, Bernard is uniquely positioned to assess whether the solution faithfully represents the method's structure and pedagogical intent — a judgment the project group could not make independently.

Overall, Bernard expressed clear satisfaction with the project's outcome. He highlighted the group's ability to deliver a coherent, functional product despite a late, significant change of direction as particularly noteworthy. The pivot from the AI-based dialog system to the structured journey was something he had supported throughout, and he confirmed in the final meeting that the shift was the right decision given both the timeframe and the nature of the method. At the same time, he acknowledged having had genuine faith in the original LLM-based approach. He noted that he remains open to revisiting a more limited AI role in future iterations of the platform — consistent with the direction outlined in chapter 11.5.

Bernard responded positively to the visual design and overall tone of the application, indicating that the calm, structured aesthetic aligns well with how Urometoden is experienced in non-digital settings. No major gaps or unmet expectations were raised. His feedback indicates that the prototype effectively conveys the method's identity and provides a credible foundation for further development.

10.0 Discussion

10.1 Did you answer the research question?

The research question from chapter 1.4: How can structured journeys be designed to deliver Urometodens content in a safe and accessible way, while maintaining user privacy?
The question has three dimensions that can be evaluated separately.
Structured digital journey: This is the most covered dimension. The delivered prototype implements an 8-week module-based journey where content is curated each week, presented in a defined order, and uses a locked progression (finished → active → locked). The data model (two-table structure: content_items and week_content, with a meta-override mechanism) enables the client to configure content per week without any code changes. The library preview provides complimentary access for users wishing to explore freely. The architecture defines how a structured journey can be formed, with a significant limitation: only the first week includes a complete content map, so the actual experience of progression throughout the eight weeks is not verified.
Safety and availability: This dimension is partially covered. The app's availability is ensured: it only requires a browser and works across devices, and the user interface is designed to be simple to navigate without guidance. Safety, in the sense of content, is strengthened by pivoting from generative AI; all content is now curated by the client, eliminating the risk of unpredictable or potentially harmful answers. But safety, in a clinical sense, is not validated: the prototype has not undergone any real user testing by the targeted audience, and we can't document that the user experience actually feels safe to people with anxiety and unrest. This limitation is significant, as the question of safety in a mental health context can't be answered through technical implementations alone; it requires empirical evaluation (De Choudhury et al., 2023).
Anonymity: This dimension is addressed through the architecture, but with a degree of tension. The pseudonymic mechanism (HMAC-SHA256-based `user_id`) and the local storage of reflection texts provide a strong foundation for anonymity. At the same time, the very existence of authentication implies (Google and email) that the system knows an identifier, even though it's hashed irreversibly. Full anonymity in an absolute sense is, for that reason, not achieved; what is achieved is pseudonymity with strong guarantees, a separation relevant in the GDPR context where pseudonymized data still counts as personal data, albeit with relaxed requirements (GDPR, 2016)
Overall, the project addresses the problem at the architectural and design levels, but not at the validation and impact levels. We have shown how such a journey can be designed; we have not shown that it works for the target group.

10.2 Reflection on the pivot

The decision to pivot from the LLM-driven dialog mechanism to a curated content journey is the most formative design decision within the project. It deserves a critical analysis, not just as a project occurrence, but as a finding with implications throughout the project.

What the pivot reveals about LLM's in a structured therapeutic context: Language models are optimized to generate the probable next token in a given context. This trait makes them well-suited for tasks where variation and adaptation are desirable, such as creative writing, code generation, and open-ended conversation. It makes them fundamentally ill-suited for tasks where consistency, predictability, and loyalty to an external structure are fundamental. Urometoden is a task like that: the content follows a set progression, the terminology is precise, and deviations from the method are not adaptations - this is wrong.

This observation is supported by a growing literature on LLM limitations in sensitive domains. (Bender et al., 2021) argues that large language models are "stochastic parrots" producing statistically plausible text without understanding the content, a trait that is especially problematic in contexts where incorrect formulations can have consequences for the user's well-being (Bender et al., 2021). (Weidinger et al., 2021) classifies risks with LLMs into categories that include harm from false information, unintentional influences on the user's mental model, and false impressions of expertise, all of which are directly relevant to a therapeutic context (Weidinger et al., 2021). (De Choudhury et al., 2023) discusses specific challenges with LLM's within mental health applications and points to generative systems that could create an illusion of empathic understanding that the user takes for granted (De Choudhury et al., 2023). Our experience confirms these warnings in practice: the model operates towards generic mindfulness-formulations, breaks with the methods' specific terminology, and cannot stay within the pedagogic progression over time, despite detailed system prompts. Prompt-based guardrails reduced the frequency of unwanted answers, but could not eliminate them. For a product where each and every answer needs to be safe, is a reduction in errors insufficient, it requires an error rate of null, which a generative model could not guarantee.

What we would do differently: In retrospect, the project group would have done things differently. The first would have carried out a faster and more systematic prototype phase for the AI-chat, with predefined test scenarios, a measurable definition of "acceptable behavior", and a clear timeframe for the concept, before investing 70% of the development time. The agile framework provided us with flexibility to pivot, but did not provide enough structure to discover the problem early. The second change would be to explore a limited AI role from the start, for example, AI-driven recommendations or summaries, rather than an open dialog, where the model operates within a tight and controlled room. This middleground is described further as possible future work in chapter 11.5.

The pivot as a design decision, not an error: It's tempting to frame the pivot as a failed attempt at an AI integration, which is erroneously framing. The project produced evidence that an open LLM dialog is unfit as a delivery mechanism for structured, therapeutic content, with value beyond the project. The result is not that AI is useless for wellness tools, but that it may not be well-suited to this specific role as an uncontrolled intermediary between the client and user. This distinction between AI as a core mechanism and AI as a support tool is a genuine contribution to the understanding of the technology's role in digital health services.

10.3 Methodology reflection

Did the agile method work for this project: In a sense, the answer is a clear yes: without the agile method, the pivot would be impossible or catastrophically costly. A waterfall project with locked requirements in the early phase would have locked the group to the AI-chat-concept throughout the entire period, and the result would have been technically

functional, but product-wise wrong. The agile approach provided us with a mechanism to absorb a fundamental change of direction, and that is exactly what agile methods are designed for (Schwaber & Sutherland, 2020).
On another level, did the project reveal a weakness in our Scrum practice: we lacked a clear acceptance criterion for the AI chat's behavior? The sprints produce functional increments, but "functional" was defined as technical (API call succeeded, answer returned) rather than behavioral (the answer is methodically correct, safe, and predictable). If we had formulated the behavior criteria early and tested them systematically, we might have discovered the problem earlier. Schwaber and Sutherland emphasize that "Definition of Done" should catch requirements of quality, not just functionality - we complied with this for technical quality, but not for content quality (Schwaber & Sutherland, 2020).
Was our feedback to the client effective? Yes, but with an important reservation. The weekly meetings with Bernard were decisive in the discovery and fulfillment of the pivot. Bernard's professional assessment of the AI chat's answers was what triggered the decision, a valuation that the project group alone did not have the competence to make. The caveat is that the feedback was reactive: Bernard responded to what we showed, rather than defining what he needed. A more structured requirements gathering process early on, for example, user histories with acceptance criteria approved by the client, could have given a clearer approach and potentially uncovered the mismatch between AI-chat and the method's needs before investing so much of the development time. Sommerville points out that stakeholder involvement is most valuable when it's proactive rather than merely evaluative (Sommerville, 2016).
What would another method change? A waterfall project would eliminate the pivot as a possibility, but also eliminate much of the uncertainty that marked the first half of the project, at the cost of worse product quality. A more exploratory approach, such as Design Thinking with a faster prototype and user testing, would potentially uncover the AI chat's limitations earlier. But within the frames of a bachelor's project with limited time and access to real users, the agile method was the most pragmatic and right choice.

10.4 Contributions to practice

Contribution to Urometoden: Before this project, Urometoden existed as a framework for individual guidance, dependent on the client's direct involvement in each course. ASHO provides the method in a digital distribution format that preserves pedagogical progression and content integrity without requiring individual guidance. The data model (week-based content mapping with positions and meta-override) enables Bernard to configure and update the journey without help from a developer. The backend infrastructure (authentication, session handling, content delivery with audio streaming) is production-ready across its core components. The project transforms Urometoden from a one-to-one service to a scalable digital platform, a prerequisite for the method to reach users beyond Bernard's personal capacity.
Contribution to digital wellness tools in general: The project's most general contribution is the LLM phase, not as a success story, but as a documented finding. We produced *practical evidence* that open LLM-based dialog is unfit as a delivery mechanism for structured, therapeutic content. This finding is relevant for developers and scientists that considers AI-integration in wellness tools, and it complements the theoretical literature

(Bender et al., 2021; Weidinger et al., 2021) with a concrete case from a real development project.
The design strategy, curated content, is a structured journey as an alternative to generative AI and is further detailed in 10.2, presenting a replicable pattern for method-specific therapeutic tools seeking digital distribution.
The project also demonstrates that anonymity and functionality are not opposites. ASHOs pseudonymizing architecture (HMAC-based identity handling, local storage of sensitive data, separate e-mail table) shows that it's possible to offer authentication, progression tracking, and content-gating without storing reversible identifying information- an approach that balances GDPR requirements, user expectations, and content need. This architectural decision can serve as a reference for other projects within health-support domains.

10.5 Limitations

The project has several specific limitations that can affect how far the conclusions can be drawn.
Incomplete content mapping: Only the first week includes complete curated content inside the database. Weeks 2-8 are structurally prepared, but the user experience of progressing through the entire journey has not been verified. This means the design choice connected to the progression, motivation, and dropout- which are central to the theoretical explanation in chapter 2.4- are currently untested in practice. Further work must be prioritized to complete content mapping and evaluate experienced progression.
No user testing: The prototype is not tested by a representative of the target group. All evaluations regarding ease of use, safety, and availability are based on the project groups and clients assumptions and professional evaluation. This is an essential limitation for an application targeted to people with anxiety and unrest -the design choice that seems appropriate for the developers without these challenges could be experienced differently by the targeted group. A structural user testing, as described in 11.4, is a precondition for further development.
Incomplete subscription flow: The trial badge and subscription model are visible in the framework, but no payment logic has been added. The backend access control to gate content based on subscription is missing. Without this flow, the business model cannot be validated, and a business model is a prerequisite for ASHO to be sustainable.
Dependent on the Cloudflare ecosystem: All infrastructure - hosting, serverless functions, database, and file-storage - is consolidated at Cloudflare. This consolidation simplifies operations and reduces integration risks, but creates a platform dependency, which means that migration to an alternative supplier would require significant work, especially for D1 (proprietary SQLite-wrapper) and the R2-integration. The application logic is written with standard web APIs to reduce the bond, but the infrastructure layer is actually locked.
Authentication system without a complete end user flow: The backend security is robust (three identity suppliers, MACH-pseudonymization, rate limiting), but the incomplete end user flow- from the first visit through registration, verification, and return to content- is not polished or user tested as a coherent experience. For the target group where technical thresholds can hinder adoption, this could be a real risk.
Statistics and progress data are hardcoded: The right panel shows statistic values for listening time, streak days, and completed days. Dynamic limitation of these values requires

an event tracking model on the server side, which is not implemented. The motivation mechanism in the framework fails without actual progress data.

11.0 Future Work

11.1 Production hardening

Rate-limiting is partially implemented for the authentication flow, but lacking in the content API's; in production, all endpoints should be protected per user and per IP. A backup strategy for the D1 database has not been established; a daily export to R2 should be implemented.
Automated testing is lacking in the codebase; device tests for API endpoints and integration tests for the authentication flow should be in place before any new functions are developed.
Occurrence procedures (rollback routines, alerts, escalation) should be formalized in a simple runbook. Betterstack is implemented for monitoring, but alerts are not configured.
Apple Sign-In is planned as a third authentication option to complement the existing Google and email/password providers. The implementation will follow standard OAuth flow with HMAC-signed state, cryptographic nonce verification, and JWT validation against Apple's JWKS endpoint. Adding Apple authentication is particularly relevant for iOS users, where Apple Sign-In is required by App Store guidelines for apps that offer third-party login.

11.2 Subscription and payment flow

The framework communicates with a subscription modal through a trial-period badge, but the backend logic remains. The API endpoints need to be expanded to include a check of subscription status (trial, active, expired, canceled) before content is returned.
Payment integration with Stripe or Vipps should follow a webhook-based model, where the supplier notifies the backend of status changes. Three new frontend screens are needed: subscription overview, payments page, and an upgrade modal for users with an expired trial period.

11.3 Content authoring tools

Check-in senere. Alt er nok på plass.

11.4 User testing and accessibility

The prototype is not tested with real users from the target group, which is the most essential limitation identified in chapter 10.5. A structured user test should be conducted with participants suffering from unrest or anxiety before the commercial release. The test should cover the core flow: first-time visit and onboarding, navigation through the weekly modules, audio player, writing in the reflections modal, and freely exploring the library. Safety and emotional comfort should be evaluated; the design choice that seems reasonable to the designers can be experienced differently by the target group. The result should inform both framework adjustments and content adaptations in consultation with Bernard.
The prototype has not been reviewed for a formal WCAG 2.1 revision. Multiple areas need

attention before production: semantic HTML and ARIA attributes are not systematically implemented, screen reader compatibility is not tested, and focus handling in modal windows (ReflectionModal, CaseModal) should be verified. Canvas-based elements (wave animation) are, by definition, unavailable to screen readers and require a text alternative.
The color coding of content types should be evaluated for color-blindness, specifically blue/purple and green/orange, and should be tested with a simulation tool to ensure they are distinctive to users with protanopia and deuteranopia.

11.5 AI reintegration

An AI integration is not excluded from future versions, but the role would be limited and controlled. Possible opportunities include personalized reflections (local, without sending data to an external service), or contextual guidance “based on the week's theme, could you try this next exercise). Common to these is that the AI operates within a tightly defined solution room defined by the client; it doesn’t generate therapeutic content, but helps the user navigate curated content. Each AI-component should be evaluated against the same criteria to uncover the problem in the original solution: unpredictability, methodical loyalty, and safety in a vulnerable context.

11.6 Observability

BetterStack is integrated for basic up-time monitoring, but the structured monitoring is missing. Three measures should be prioritized. A structured login with correlation IDs per request, to ensure the user journey can be traced from the client through an API to the database, for troubleshooting. Cost-monitoring per Cloudflare service (workers-invocations, D1-readings, R2-bandwidth), when serverless functions can generate unexpected costs with a traffic stopper. Alerts with error-rates over threshold values, connection between BetterStack-alerts and a defined escalations routine as described in 11.1.

12.0 Conclusion

The goal for this project was to develop a digital platform that made Urometoden available as a structured, safe, and anonymous 8-week journey. The problem asked how such a journey could be designed to deliver the method's content safely while preserving the user's anonymity.

The project delivers a functional prototype that answers these questions at the architecture and design level. This structured journey is implemented with a data model that supports weekly-based content curation, locked progression, and four content types (audio, case, reflection, video). Audio streaming with seeking, full-screen player with a wave animation, reflection modules with local storage, and a visual timeline over the eight weeks are in place. The library provides users with free access to all content as a complimentary access path. The backend infrastructure - authentication with three identity suppliers, HMAC-pseudonymized user identity, parameterized SQL, multi-layered validations, and audio streaming through R2 - is production-ready within its core components.

The project delivers a platform that preserves Urometoden's pedagogical progression and content integrity in digital form, without requiring individual guidance. The data models' meta-override mechanism makes it possible to configure the content per week without developer assistance - a precondition for long-term sustainability.

The project delivers documented findings: open LLM-based dialog is unfit as a delivery mechanism for structured, therapeutic content. This insight, achieved through 70% of the development time being invested in the AI-chat concept, led to a pivot which resulted in a better product, and as relevance exceeded this project for others considering AI-integration in health-related tools.

Significant limitations still remain. Only week 1 has complete content mapping. The prototype is not user-tested by the target group. The subscription flow and dynamic progression tracking are not implemented. These limitations define the natural continuation of the work, described in chapter 11.

The project has shown how a structured digital journey for Urometoden can be designed. It remains to be shown that it works through complete content mapping, user testing, and commercial launch.

Bibliography

Andrade, L. H., Alonso, J., Mneimneh, Z., Wells, J. E., Al-Hamzawi, A., Borges, G., Bromet, E., Bruffaerts, R., de Girolamo, G., de Graaf, R., Florescu, S., Gureje, O., Hinkov, H. R., Hu, C., Huang, Y., Hwang, I., Jin, R., Karam, E. G., Kovess-Masfety, V., … Kessler, R. C. (2014). Barriers to Mental Health Treatment: Results from the WHO World Mental Health (WMH) Surveys. Psychological Medicine, 44(6), 1303–1317. https://doi.org/10.1017/S0033291713001943

Authentication—OWASP Cheat Sheet Series. (n.d.). Retrieved May 7, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Baumel, A., Muench, F., Edan, S., & Kane, J. M. (2019). Objective User Engagement With Mental Health Apps: Systematic Search and Panel-Based Usage Analysis. Journal of Medical Internet Research, 21(9), e14567. https://doi.org/10.2196/14567

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜. Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, FAccT '21, 610–623. https://doi.org/10.1145/3442188.3445922

Cross-Site Request Forgery Prevention—OWASP Cheat Sheet Series. (n.d.). Retrieved May 7, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html#typescript-utilities-for-csrf-protection

Datatilsynet. (n.d.). Datatilsynet. Retrieved May 7, 2026, from https://www.datatilsynet.no/rettigheter-og-plikter/virksomhetenes-plikter/vurdering-av-personvernkonsekvenser/

De Choudhury, M., Pendse, S. R., & Kumar, N. (2023). Benefits and Harms of Large Language Models in Digital Mental Health. PsyArXiv. https://doi.org/10.31234/osf.io/y8ax9

Eriksen, M. (2024, July). Leveraging the Psychology of Color in UX Design for Health and Wellness Apps. https://www.uxmatters.com/mt/archives/2024/07/leveraging-the-psychology-of-color-in-ux-design-for-health-and-wellness-apps.php

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures [PhD dissertation, University of California]. https://roy.gbiv.com/pubs/dissertation/fielding_dissertation_2up.pdf

Fielding, R. T., Lafon, Y., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Range Requests (Request for Comments RFC 7233). Internet Engineering Task Force. https://doi.org/10.17487/RFC7233

Fitzpatrick, K. K., Darcy, A., & Vierhile, M. (2017). Delivering Cognitive Behavior Therapy to Young Adults With Symptoms of Depression and Anxiety Using a Fully Automated Conversational Agent (Woebot): A Randomized Controlled Trial. JMIR Mental Health, 4(2), e7785. https://doi.org/10.2196/mental.7785

Fogg, B. (2009). A behavior model for persuasive design. Proceedings of the 4th International Conference on Persuasive Technology, 1–7. https://doi.org/10.1145/1541948.1541999

Hardt, D. (2012). The OAuth 2.0 Authorization Framework (Request for Comments RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research.

Initiative (WAI), W. W. A. (2024). Cognitive Accessibility at W3C. Web Accessibility Initiative (WAI). https://www.w3.org/WAI/cognitive/

Krawczyk, H., Bellare, M., & Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication (Request for Comments RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

Lov om behandling av personopplysninger (personopplysningsloven)—Lovdata. (2018). https://lovdata.no/dokument/NL/lov/2018-06-15-38

Meta Open Source. (n.d.). React Documentation. Retrieved May 7, 2026, from https://react.dev/

OWASP Top 10:2025. (n.d.). Retrieved May 7, 2026, from https://owasp.org/Top10/2025/

Password Storage—OWASP Cheat Sheet Series. (n.d.). Retrieved May 7, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Payne, P., Levine, P. A., & Crane-Godreau, M. A. (2015). Somatic experiencing: Using interoception and proprioception as core elements of trauma therapy. Frontiers in Psychology, 6. https://doi.org/10.3389/fpsyg.2015.00093

Principles behind the Agile Manifesto. (n.d.). Retrieved May 6, 2026, from https://agilemanifesto.org/iso/en/principles.html

Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data, and Repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA Relevance), 119 OJ L (2016). http://data.europa.eu/eli/reg/2016/679/oj

Roy D. Pea. (2004). The Social and Technological Dimensions of Scaffolding and Related Theoretical Concepts for Learning, Education, and Human Activity. https://web.stanford.edu/~roypea/RoyPDF%20folder/A117_Pea_04_JLS_Scaffolding.pdf

Schwaber, K., & Sutherland, J. (2020). Scrum Guide | Scrum Guides. https://scrumguides.org/scrum-guide.html

Seip, C., & Bøhmer, B. (2014). Kampen mot uroen (1st ed.). Gyldendal.

Sommerville, I. (2016). Software engineering (Tenth edition). Pearson.

SQL Injection Prevention—OWASP Cheat Sheet Series. (n.d.). Retrieved May 7, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Web Content Accessibility Guidelines (WCAG) 2.1. (2025). https://www.w3.org/TR/WCAG21/

Weidinger, L., Mellor, J., Rauh, M., Griffin, C., Uesato, J., Huang, P.-S., Cheng, M., Glaese, M., Balle, B., Kasirzadeh, A., Kenton, Z., Brown, S., Hawkins, W., Stepleton, T., Biles, C., Birhane, A., Haas, J., Rimell, L., Hendricks, L. A., … Gabriel, I. (2021). Ethical and social risks of harm from Language Models (arXiv:2112.04359). arXiv. https://doi.org/10.48550/arXiv.2112.04359

Appendix

Expanded risk document

Appendix A: Database Schema (schema.sql + seed.sql)

Appendix B: AI-chat pipeline - the seven steps, prompt-design, guardrails, theme-configuration.
Each message went through a structured pipeline before an answer was returned. (1) security validation with NFC-normalization, symbol allowlist, token counting and regex-based heuristics against prompt-injection; (2) idempotent check on (conversation_id, mesage_id) to avoid double execution against LLM; (3) token-budget that is automatically pulled in Postgres before the call; (4) theme classifier via embedding-conscious equality which decides which system prompt variant is used; (5) context window-handling where older messages are summarized and replaced with a summary when the token-limit is exceeded (6) LLm-call with a composed prompt; (7) persistence of both user and assistant messages and daily token usage.

The prompt design is divided into configurable themes stored in the database, each with its own systemprompt, microinstructions, pacing-rules, security rules and re-classification threshold. Guardrails includes per-message token ceiling, session budget, injection-heuristics, logging all rejections, admin only endpoints and generic 500-answers that never leaks internal errors to the user.

Appendix C: Input validation- Pydantic field lengths, character allowlist, prompt-injection heuristics, token cap

The first layer is form validation via Pydantic, which enforces field length and presence: e-mail is limited to 3-320 characters, password to 8-1024, reset-token to 12-512, and chat-field requires a minimum of 1 character. The second layer is unicode-normalization to NFC-form, trimming of whitespace and collapsing of repeated spacing. The third layer is a hard character allowlist which only allows ASCII-letters, Norwegian letters (æøåÆØÅ), latin letters with diacritical characters (verified through unicodedata.name), combining symbols, numbers and a limited set of punctuation. Everything else including JSON special characters, backslash, angle brackets, @, ; and $ are denied. The fourth layer is a token-cap per message against Max_Message_Tokens (default 512). The fifth layer is prompt-injection-heuristics which flags known attack patterns as “ignore previous instructions”, “system prompt”, “jailbreak” and such, and return HTTP 400 on hit.

Idempotency on the combination (conversation_id, message_id) are secured through a SHA-256-hash of normalized payload. This blocks both reuse of the same message_id with different content (409 conflict) and that identical requests are double processed.

SQL-injections are prevented throughout with the use of parameterized SQL (%s -placeholders via psycopg). There are no f-strings or string concatenation inside SQL-sentences inside authentications, or the chat-code, the strongest defense mechanism

against SQL -injections according to OWASP (*SQL Injection Prevention - OWASP Cheat Sheet Series*, n.d.).

Cross-site scripting (XSS) is primarily a risk frontend. React 18 escapes as standard all values set in JSX, which provides good protection against reflected and stored XSS (Meta Open Source, n.d.). No user input is rendered as raw HTML (dangerouslySetInnerHTML is not used). Reflection texts are stored and read from localStorage without being injected in DOM as unescaped HTML.